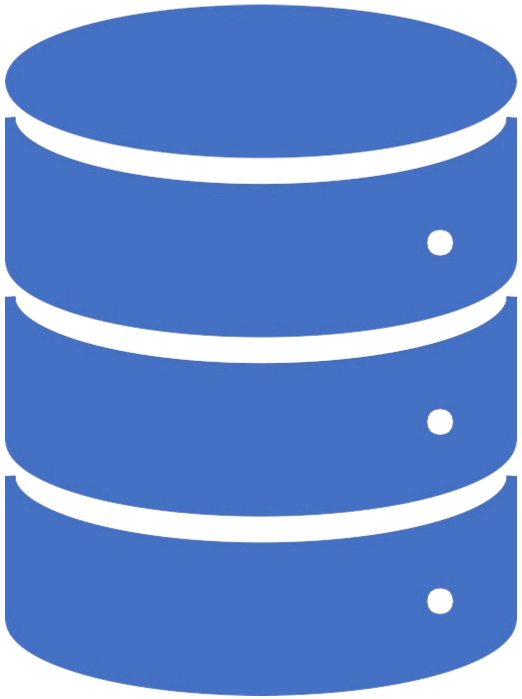


Database System
[*Keys in DBMS*]

Keys

- Keys play an important role in the relational database.
- It is used to uniquely identify any record or row of data from the table. It is also used to establish and identify relationships between tables.
- For example, `ID` is used as a key in the `STUDENT` table because it is unique for each student.
- In the `PERSON` table, `passport_number`, `license_number`, `SSN` are keys since they are unique for each person.

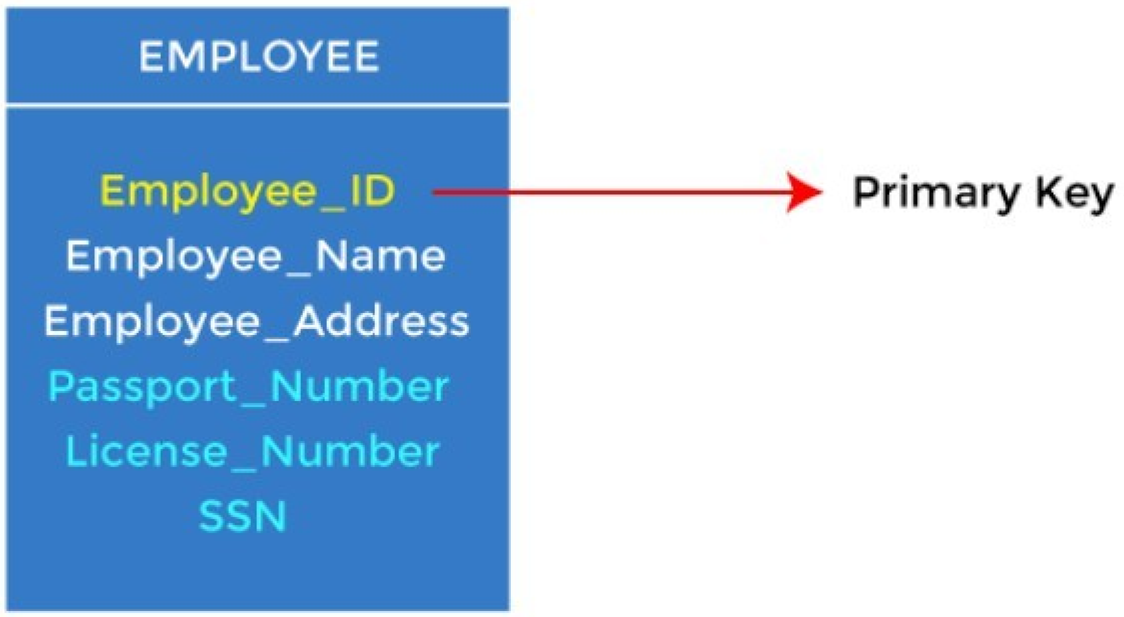
Types of Keys

- Primary Key
- Candidate Key
- Super Key
- Foreign Key
- Alternate Key
- Composite/ Concatenate Key
- Artificial Key
- Secondary Key
- Sort/ Control Key

Primary Key

- It is the ***key used to identify one and only one instance of an entity uniquely***.
- An entity can contain multiple keys, as we saw in the PERSON table.
- The key which is most suitable from those lists becomes a primary key.
- Primary Key ***cannot contain null*** values and a relation can have only ***one primary key***.

Primary Key



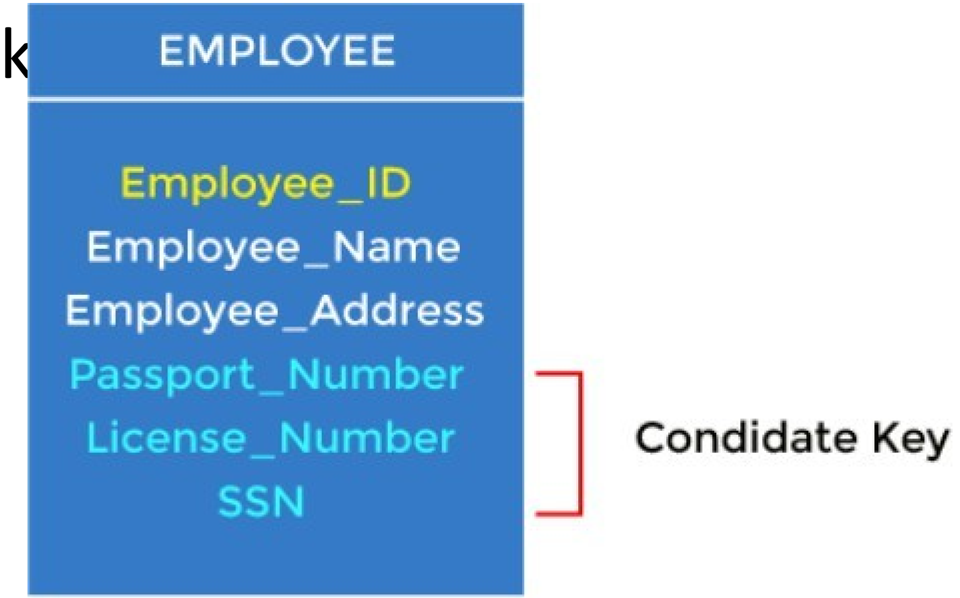
- In the **EMPLOYEE** table, **Employee_ID** can be the ***primary key*** since it is unique for each employee.
- In the **EMPLOYEE** table, we can even select License_Number and Passport_Number as primary keys since they are also unique.
- For each entity, the primary key selection is based on requirements and developers.

Candidate Key

- A candidate key is an ***attribute or set of attributes that can uniquely identify a tuple OR attribute(set of attributes) that can be used as primary key***.
- Except for the primary key, the remaining attributes are considered a candidate key.
- The candidate keys are as strong as the primary key.

Candidate Key Example

- In the **EMPLOYEE** table, `Employee_ID` is best suited for the primary key.
- The rest of the attributes, like `SSN`, `Passport_Number`, `License_Number`, etc., are considered a candidate k

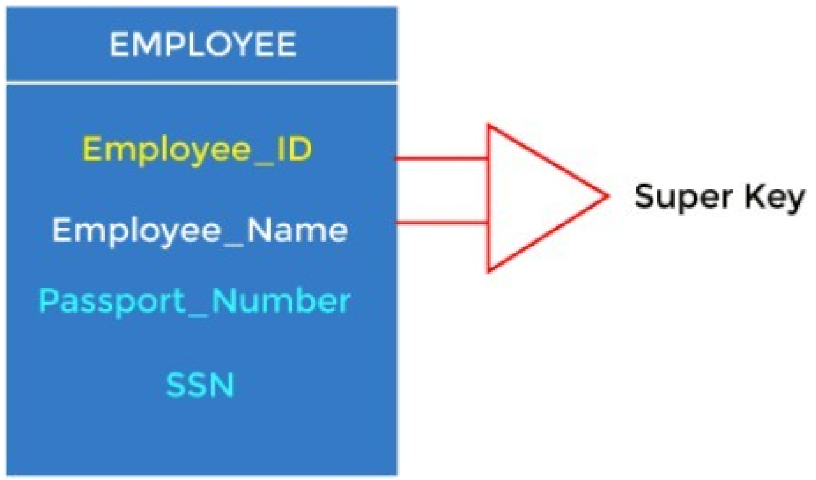


Super Key

- Super key is an ***attribute set that can uniquely identify a tuple***.
- A super key is a ***superset of a candidate key***.

Super Key Example

- In the **EMPLOYEE** table, for (`Employee_ID`, `Employee_Name`), the name of two employees can be the same, but their `Employee_ID` can't be the same. Hence, this combination can also be a key.
- The super key would be `Employee_ID` (`Employee_ID`, `Employee_Name`), etc.

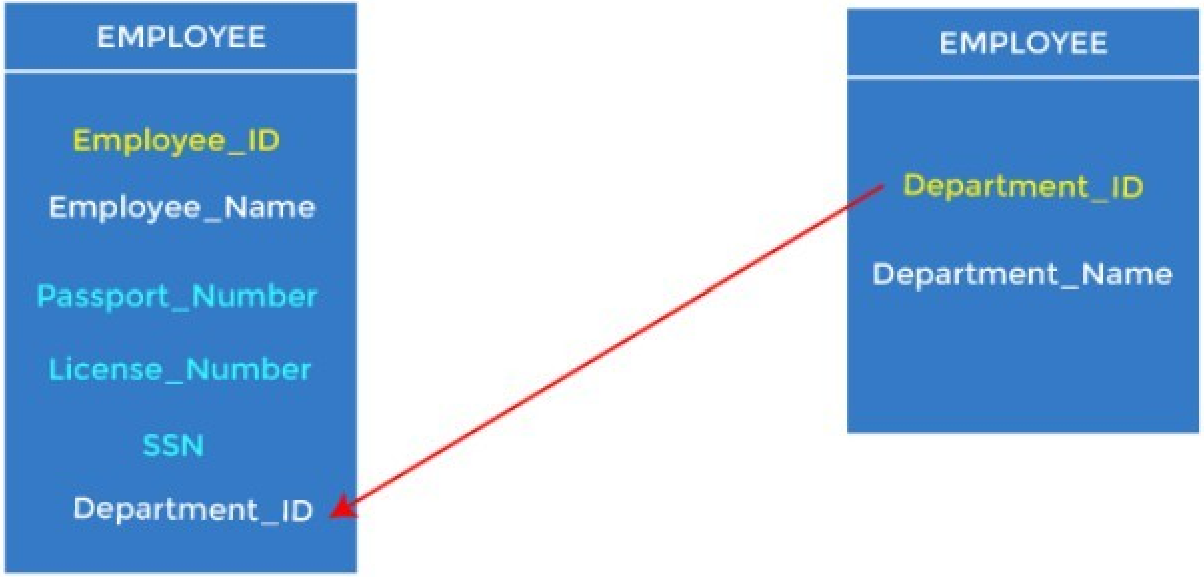


Foreign Key

- Foreign keys are ***the column of the table used to point to the primary key of another table***.
- Every employee works in a specific department in a company, and employee and department are two different entities.
- So we can't store the department's information in the employee table. That's why we link these two tables through the primary key of one table.

Foreign Key Example

- We add the primary key of the **DEPARTMENT** table, `Department_ID`, as a new attribute in the **EMPLOYEE** table.
- In the **EMPLOYEE** table, `Department_ID` is the foreign key, and both the tables are



Foreign Key

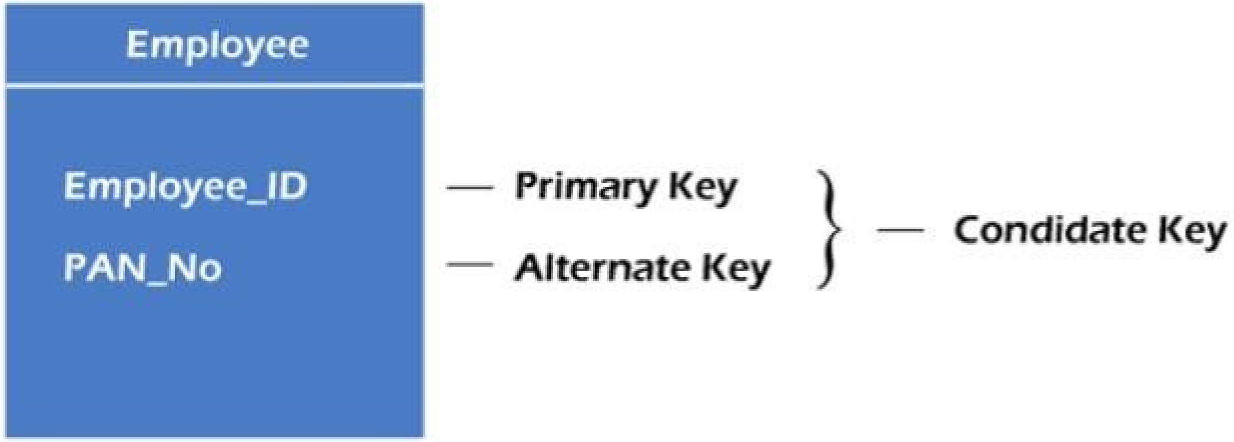
- The relation in which foreign key is created is known as **dependent relation** or **child relation**.
- The relation to which the foreign key refers is known as **parent relation**.

Alternate Key

- There may be one or more attributes or a combination of attributes that uniquely identify each tuple in a relation. These attributes or combinations of the attributes are called the candidate keys. One key is chosen as the primary key from these candidate keys, and the remaining candidate key, if it exists, is termed the ***alternate key***.
- In other words, ***the total number of the alternate keys is the total number of candidate keys minus the primary key***.
- The alternate key may or may not exist. If there is only one candidate key in a relation, it does not have an alternate key.

Alternate Key Example

- **EMPLOYEE** relation has two attributes, `Employee_ID` and `PAN_No`, that act as candidate keys.
- In this relation, `Employee_ID` is chosen as the primary key, so the other candidate key, `PAN_No`, acts as the Alternate key.

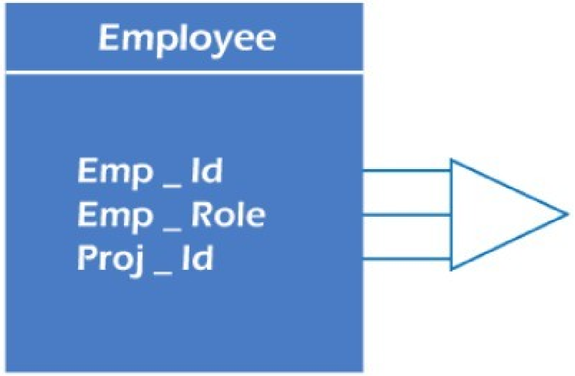


Composite/ Concatenate Key

- Whenever ***a primary key consists of more than one attribute, it is known as a composite key***.
- This key is also known as ***Concatenated Key***.

Composite Key Example

- In **EMPLOYEE** relations, we assume that an employee may be assigned multiple roles, and an employee may work on multiple projects simultaneously.
- So the primary key will be composed of all three attributes, namely `Emp_Id`, `Emp_Role`, and `Proj_Id` in combination.
- So these attributes act as a composite key since the primary key comprises more than one attribute.

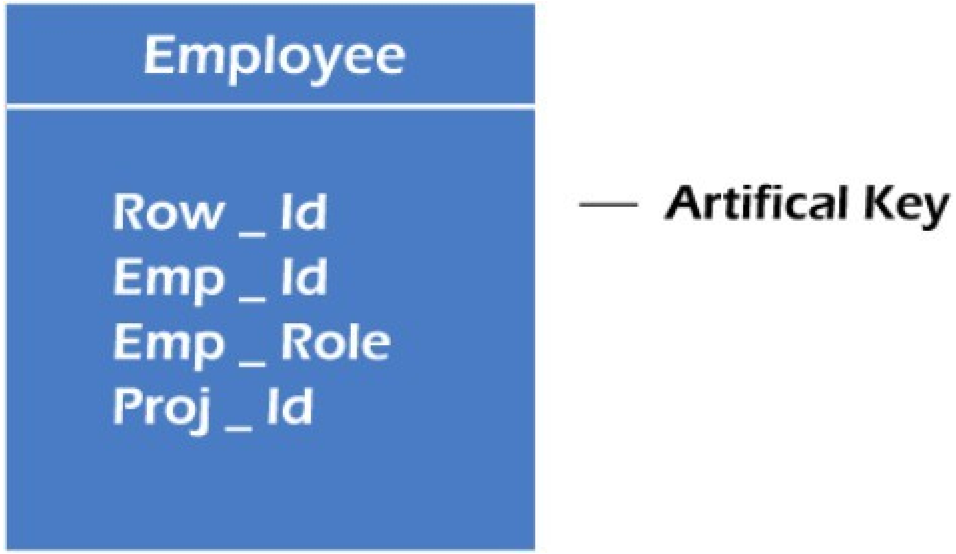


Artificial Key

- The ***key created using arbitrarily assigned data are known as artificial keys***.
- These keys are created when a primary key is large and complex and has no relationship with many other relations.
- The data values of the artificial keys are usually numbered in a serial order.

Artificial Key Example

- The primary key, which is composed of `Emp_Id`, `Emp_Role`, and `Proj_Id`, is large in **EMPLOYEE** relations.
- So it would be better to add a new virtual attribute to identify each tuple in the relation uniquely.



Secondary Key

- A **secondary key** is an attribute or combination of attributes that can be used to access or retrieve records.
- Secondary key values may not be unique.
- One secondary key value may refer to many records.
  - For example, an attribute **City** in **Student** relation can be used to display all students who live in a specific city. In this case, **City** is used as secondary key.

Sort/ Control Key

- An attribute or set of attributes that is used to physically sequence the stored data is called **sort key**.
- It is also known as **control key**.
- The stored data can be stored in different ways according to the user requirement.